

PROJECT REPORT: SmartDine-Pro Platform

Date: April 2026 **Author:** Karthik **Platform:** Multi-Page Web Application (React, Vite, Firebase)

1. Executive Summary

SmartDine-Pro is a fully functional, cloud-synchronized restaurant management and self-ordering ecosystem. Designed to mitigate traditional front-of-house bottlenecks, it enables customers to self-order using a mobile Progressive Web App (PWA) via QR codes, while simultaneously dispatching tickets instantly to a cloud-synced Kitchen Waiter display and a Master Management Billing console.

2. System Architecture

The application shifted from a legacy architecture to a modern Multi-Page Application (MPA) using Vite. Instead of forcing heavy code onto every device, the system is rigidly split into three optimized silos:

- index.html (Customer UX):** Lightweight menu display and cart. Optimized for mobile 3G/4G connections.
- staff.html (Management):** Heavy data processing, revenue calculators, daily spreadsheet export mechanisms.
- waiter.html (Service):** Pure operational status tracking with real-time UI flashes for new/ready orders.

Tech Stack Utilized

- Core Framework:** React 18, TypeScript, Vite
 - Styling Engine:** Vanilla CSS Modular standard (Glassmorphism & UX Animations)
 - Database Architecture:** Google Firebase Firestore
 - Deployment Platform:** Google Firebase Hosting (Globally scalable CDN edge locations)
-

3. Key Implementations

Mobile-First Customer Flow

- QR Table Binding:** Customers scan a table-specific QR code, bypassing generic login screens.
- Mandatory Authentication:** Requires Customer Name & Mobile Number, ensuring table integrity.
- Cart & UPI Integration:** Includes an end-of-flow prompt allowing the customer to initiate UPI payments directly, or opt for counter-pay.

Kitchen & Waiter Subsystem

- Real-time onSnapshot listeners bind directly to Firebase. When an order fires, the Waiter Screen blinks the new ticket instantly without refreshing.
- Statuses advance via one-touch logic (Queued > Cooking > Ready).

Master Billing & Admin Console

- PIN Authorization Gate:** Restricted by a system-level PIN (5258).
- Live Menu Pricing:** Administrators can inline-edit item prices. Saves are pushed automatically back into the Firebase JSON tree and update customer phones live.

- **Excel Subroutine:** A custom-built algorithm translates JSON objects of the day's revenue and operations into universally parsable Comma Separated Values (.csv) for Excel ingestion.

4. Live Access URLs (Production)

These are the permanent links to access your live infrastructure. Save and distribute these to your staff.

- **Customer Menu (Table 1 Example):** <https://smartdine-pro.web.app/table/1>
- **Customer Menu (Table 5 Example):** <https://smartdine-pro.web.app/table/5>
- **Kitchen / Chef Display:** <https://smartdine-pro.web.app/staff.html#/kitchen> (PIN: 5258)
- **Management Console:** <https://smartdine-pro.web.app/staff.html#/billing> (PIN: 5258)
- **Waiter Service Panel:** <https://smartdine-pro.web.app/waiter.html> (Unrestricted access for fast-floor usage)

5. Execution & Operating Procedures

(Steps for standard administrative operations)

A. Local Testing & Development

To make changes to code safely:

1. Open terminal inside the SmartDine-Pro folder.
2. Execute `npm run dev` to boot up the local Node Server.
3. Test locally at `http://localhost:5173`. When finished testing, close terminal with `Ctrl + C`.

B. Deploying to the Internet (Production)

To push changes to the live internet domain:

1. Execute `npm run build` — Vite will aggressively minify, compress, and chunk the codebase into the `/dist` folder.
2. Execute `npx firebase-tools deploy` — Pushes the `/dist` bundle securely onto Google's servers.
3. *Note: The Firebase Hosting pipeline uses the `firebase.json` file which defines "rewrites" to guarantee SPAs do not return 404 errors.*

C. Resetting the Application (End of Day)

1. Navigate to the live Billing Console.
2. Click **Export to Excel** to backup the daily revenue.
3. Click the red **Clear All Orders** button to truncate the Firebase database queue for the next day.
4. Close your computer. The Cloud handles the 24/7 uptime automatically.

5. Security Summary

- **No Password Management:** User accounts are scoped purely via session storage mitigating severe GDPR/PII leakage liability.
- **Hosting URL Constraints:** Internal Waiter and Staff pages are decoupled from the main `.web.app` root, ensuring users cannot easily guess administrative paths, supplemented by PIN-checks.

End of Report.